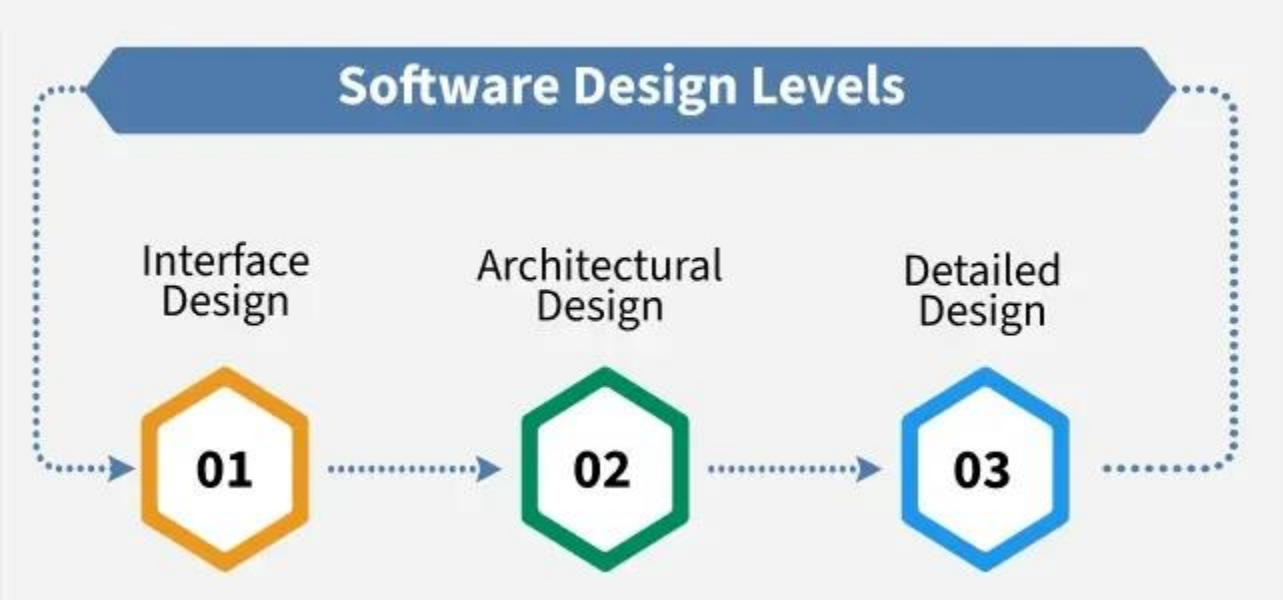
3.Design Engineering

What is Software Design Process?

Software Design Process is the phase where developers plan how to turn a set of requirements into a working system. Like a blueprint for the software. Instead of going straight into writing code, developers break down complex requirements into smaller, manageable pieces, design the system architecture, and decide how everything will fit together and work.
The software design process can be divided into the following three levels or phases of design:



1.Creating an architectural design
2.Design concepts
3.Software Architecture
4.Data design
5.Architectural styles and patterns
6.Architectural Design

1. Design Process and Design Quality

- **Design Process**: A series of steps to convert requirements into a blueprint for building software.
    - Steps: Requirements → Analysis → Design → Implementation → Testing
- **Design Quality**: Good design is:
    - Correct, efficient, reliable, maintainable, and reusable.

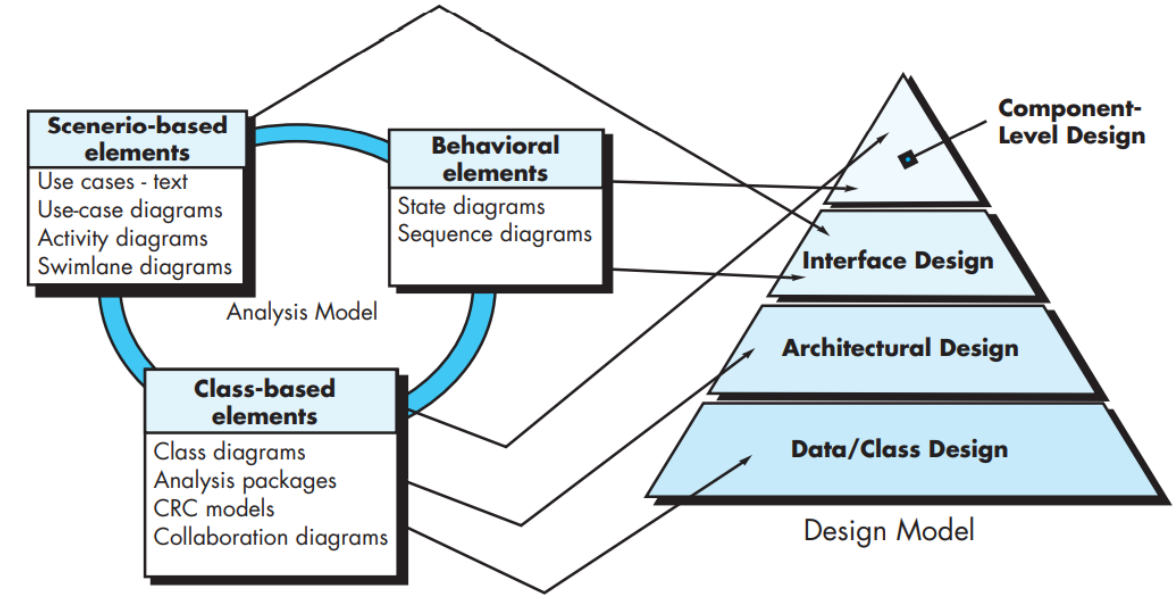
2. Design Concepts

- **Abstraction**: Hiding details to focus on high-level view.
- **Modularity**: Dividing system into manageable components.
- **Cohesion**: Degree of relatedness within a module.
- **Coupling**: Degree of interdependence between modules (low is better).
- **Encapsulation**: Hiding internal details of a module.

3. The Design Model

- Consists of:
    - **Data Design**: How data is structured.
    - **Architectural Design**: Overall structure of the system.
    - **Interface Design**: How modules interact.
    - **Component Design**: Internal design of each module.
    - **Deployment Design**: How software runs on hardware.

FIGURE 12.1 Translating the requirements model into the design model



Interface Design

Interface Design is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a black box. Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called agents.

Interface design should include the following details:

1. Precise description of events in the environment, or messages from agents to which the system must respond.
2. Precise description of the events or messages that the system must produce.
3. Specification of the data, and the formats of the data coming into and going out of the system.
4. Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

Architectural Design

Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored. Issues in architectural design includes:

1. Gross decomposition of the systems into major components.
2. Allocation of functional responsibilities to components.
3. Component Interfaces.
4. Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
5. Communication and interaction between components.

The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

Detailed Design

Detailed design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures. The detailed design may include:

1. Decomposition of major system components into program units.
2. Allocation of functional responsibilities to units.
3. User interfaces.
4. Unit states and state changes.
5. Data and control interaction between units.
6. Data packaging and implementation, including issues of scope and visibility of program elements.
7. Algorithms and data structures.

Software Design Phases

The Actual Software Design process travel through the one end to another end of making and Software well with completion of requirements and the phases which included in the same that are bellow:

Software Quality Guidelines:

A design should exhibit an architecture that (1) has been created using recognizable architectural styles or patterns, (2) is composed of components that exhibit good design characteristics and (3) can be implemented in an evolutionary fashion, thereby facilitating implementation and testing.

A design should be modular; that is, the software should be logically partitioned into elements or subsystems.

A design should contain distinct representations of data, architecture, interfaces, and components.

A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

A design should lead to components that exhibit independent functional characteristics.

6. A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.

A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.
A design should be represented using a notation that effectively communicates its meaning.

Quality Attributes

a set of software quality attributes that has been given the acronym FURPS

Functionality
usability
reliability
performance
supportability.

The FURPS quality attributes represent a target for all software design:

These attributes represent a more common term, maintainability— and in addition, testability, compatibility, configurability the ease with which a system can be installed, and the ease with which problems can be localized.

THE DESIGN PROCESS

Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software. Initially, the blueprint depicts a holistic view of software.

Design Concepts

A set of fundamental software design concepts has evolved over the history of software engineering.
Each helps you answer the following questions:
What criteria can be used to partition software into individual components?
How is function or data structure detail separated from a conceptual representation of

the software?
What uniform criteria define the technical quality of a software design?

- Abstraction
- Architecture
- Patterns
- Separation of Concerns
- Modularity
- Information Hiding
- Functional Independence
- Refinement
- Re-factoring
- Aspects
- Design Classes

ABSTRACTION

Many levels of abstraction

Highest level of abstraction : Solution is stated in broad terms using the language of the problem environment

Lower levels of abstraction : More detailed description of the solution is provided

Procedural abstraction:

-- Refers to a sequence of instructions that a specific and limited function

An example of a procedural abstraction would be the word open for a door.

Open implies a long sequence of procedural steps (e.g., walk to the door, reach out and grasp knob, turn knob and pull door, step away from moving door, etc.)

Data abstraction:

-- Named collection of data that describe a data object

The data abstraction for door would encompass a set of attributes that describe the door (e.g., door type, swing direction, opening mechanism, weight, dimensions).

ARCHITECTURE

Software architecture is the structure or organization of program components (modules), the manner in which these components interact, and the structure of data that are used by the components.

Shaw and Garlan [Sha95a] describe a set of properties that should be specified as part of an architectural design:

Structural properties: e.g., modules, objects, filters

Extra-functional properties: how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

Families of related systems.

Architecture Models

(a) Structural Models

-- An organized collection of program components

(b) Framework Models

-- Represents the design in more abstract way

(c) Dynamic Models

-- Represents the behavioral aspects indicating changes as a function of external events

(d) Process Models

-- Focus on the design of the business or technical process

(e) Functional models

--can be used to represent the functional hierarchy of a system.

Design Concepts

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice." --Christopher Alexander

- Provides a description to enables a designer to determine the followings :
  (a) Whether the pattern is applicable to the current work
  (b) Whether the pattern can be reused
  (c) Whether the pattern can serve as a guide for developing a similar but functionally or structurally different pattern

Separation of Concerns

Separation of concerns is a design concept [Dij82] that suggests that any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently.

A concern is a feature or behavior that is specified as part of the requirements model for the software.

MODULARITY

Divides software into separately named and addressable components, sometimes called modules
Modules are integrated to satisfy problem requirements
Consider two problems p1 and p2. If the complexity of p1 is cp1 and of p2 is cp2 then effort to solve p1=cp1 and effort to solve p2=cp2
If cp1>cp2 then ep1>ep2
The complexity of two problems when they are combined is often greater than the sum of the perceived complexity when each is taken separately
Based on Divide and Conquer strategy : it is easier to solve a complex problem when broken into sub-modules

INFORMATION HIDING

Information contained within a module is inaccessible to other modules who do not need such information
Achieved by defining a set of Independent modules that communicate with one another only that

information necessary to achieve software function
Provides the greatest benefits when modifications are required during testing and later
Errors introduced during modification are less likely to propagate to other location within the software

FUNCTIONAL INDEPENDENCE

It is achieved by design a software that each module address a specific sub function of requirements and has a simple interface when viewed from other parts of program structure
Why independence is so important?
=> Independent modules are easier to maintain
=> Error propagate is reduced
=> Reusable modules are possible

Independence is assessed by two quantitative criteria:
Cohesion
-- Performs a single task requiring little interaction with other components
Coupling
--Measure of interconnection among modules
Coupling should be low and cohesion should be high for good design

REFINEMENT & REFACTORING

Refinement
Process of elaboration from high level abstraction to the lowest level abstraction.
High level abstraction begins with a statement of functions.
Refinement causes the designer to elaborate providing more and more details at successive level of abstractions.
Abstraction and refinement are complementary concepts.

Refactoring
Organization technique that simplifies the design of a component without changing its function or behavior.
Examines for redundancy, unused design elements and inefficient or unnecessary algorithms.

Aspects

As requirements analysis occurs, a set of “concerns” is uncovered. These concerns include “requirements, use cases, features, data structures, quality-of-service issues, variants, intellectual property boundaries, collaborations, patterns and contracts”.
An aspect is a representation of a crosscutting concern.
A crosscutting concern is some characteristic of the system that applies across many different

requirements.

DESIGN CLASSES

Class represents a different layer of design architecture.
Five types of Design Classes
1. User interface class
   -- Defines all abstractions that are necessary for human computer interaction
2. Business domain class
   -- Refinement of the analysis classes that identity attributes and services to implement some of business domain
3.Process class
   -- implements lower level business abstractions required to fully manage the business domain classes
4.Persistent class
   -- Represent data stores that will persist beyond the execution of the software
5.System class
   -- Implements management and control functions to operate and communicate within the computer environment and with the outside world.

Architectural Design

Software Architecture
Architectural Styles

What Is Architecture?

"The architecture of a system is a comprehensive framework that describes its form and structure—its components and how they fit together."
--Jerrold Grochow
The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.
--Bass, Clements, and Kazman [Bas03]

Software Architecture is not the operational software. It is a representation that enables a software engineer to
Analyze the effectiveness of the design in meeting its stated requirements.
consider architectural alternative at a stage when making design changes is still relatively easy .

Reduces the risk associated with the construction of the software.

Why Is Architecture Important?

Three key reasons
Representations of software architecture enables communication and understanding between stakeholders.
Highlights early design decisions to create an operational entity.
Constitutes a model of software components and their interconnection.

Architectural styles

Taxonomy of Architectural Styles
Data-centered architectures
Data-flow architectures
Call and return architectures
Object-oriented architectures
Layered architectures
Definition: An architectural style is a transformation that is imposed on the design of an entire system. The intent is to establish a structure for all components of the system.
Each styles describes a system category that encompasses:
a set of components
a set of connectors that enables “communication and coordination
Constraints that define how components can be integrated to form the system
Semantic models to understand the overall properties of a system

Data-centered architectures

Data-centered architectures promote integrability [Bas03]. That is, existing components can be changed and new client components added to the architecture without concern about other clients (because the client components operate

independently).

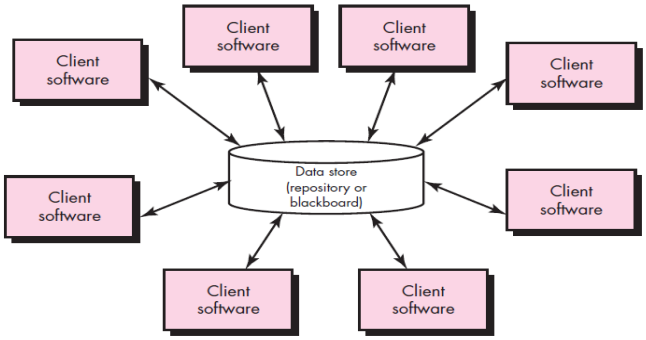


Figure: Data-centered architecture

Data-flow architectures

Shows the flow of input data, its computational components and output data
Structure is also called pipe and Filter
Pipe provides path for flow of data
Filters manipulate data and work independent of its neighboring filter
If data flow degenerates into a single line of transform, it is termed as batch sequential.

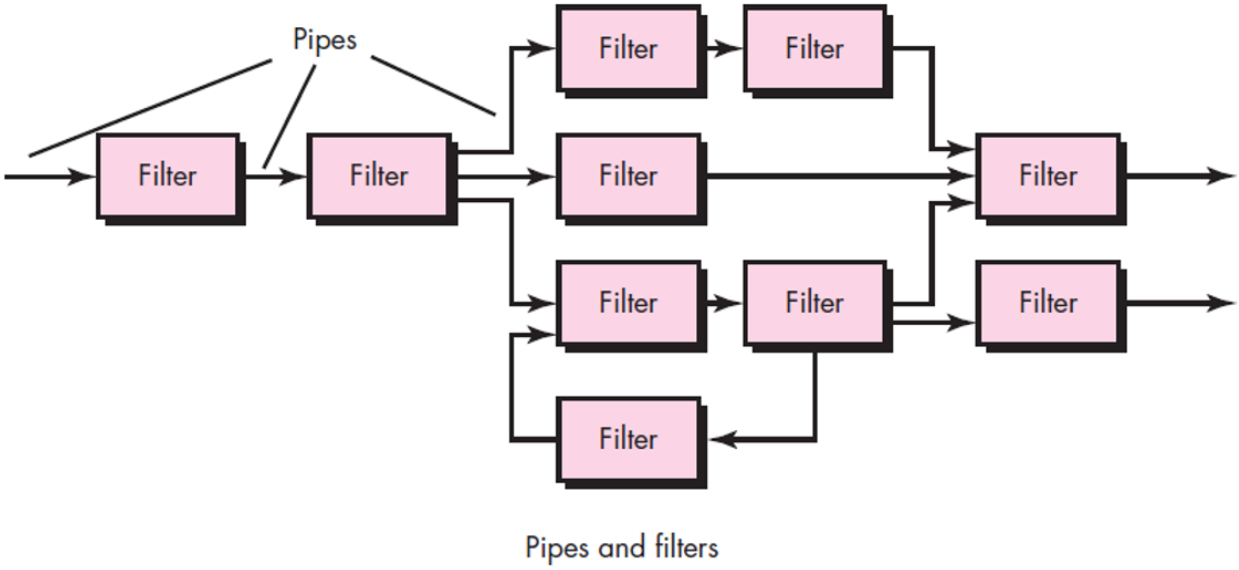


Figure: Data-flow architecture

Call and return architectures

Achieves a structure that is easy to modify and scale

Two sub styles

(1) Main program/sub program architecture

-- Classic program structure

-- Main program invokes a number of components, which in turn invoke still other components

(2) Remote procedure call architecture

--Components of main program/subprogram are distributed across computers over network

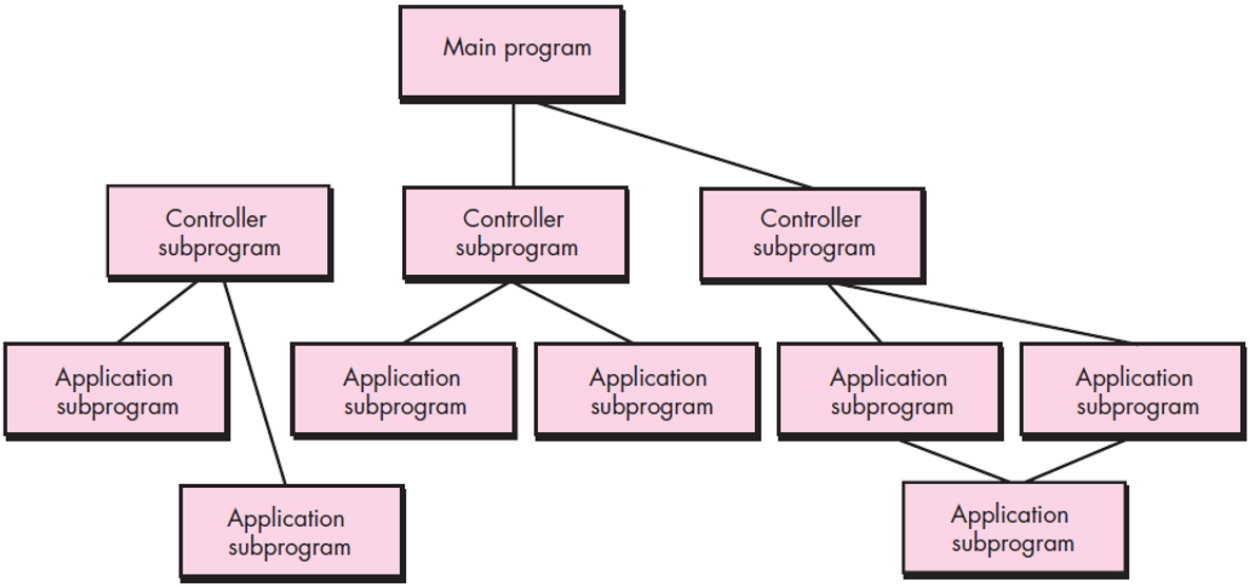


Figure: Main program/subprogram architecture

Object-oriented architectures

The components of a system encapsulate data and the operations

Communication and coordination between components is done via message

Layered architectures

A number of different layers are defined

Inner Layer(interface with OS)
Intermediate Layer (Utility services and application function)
Outer Layer (User interface)

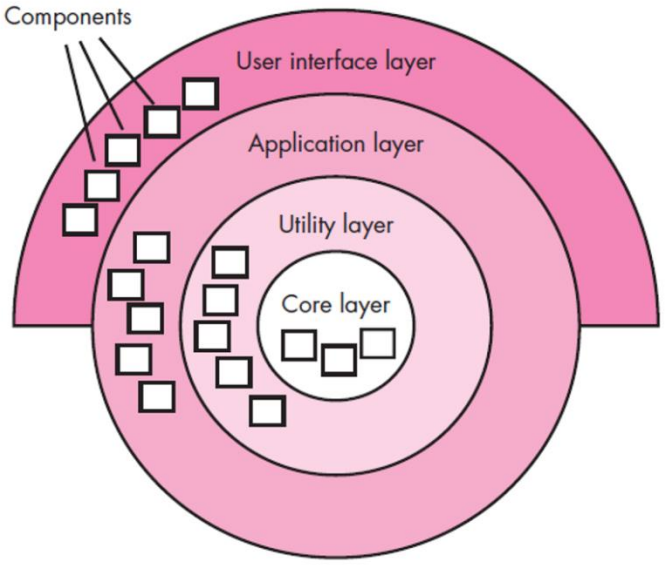


Figure: Layered Architecture

Architectural Design

As architectural design begins, the software to be developed must be put into context—that is, the design should define the external entities (other systems, devices, people) that the software interacts with and the nature of the interaction. An archetype is an abstraction (similar to a class) that represents one element of system behavior.

Representing the System in Context
Defining Archetypes
Refining the Architecture into Components

Describing Instantiations of the System

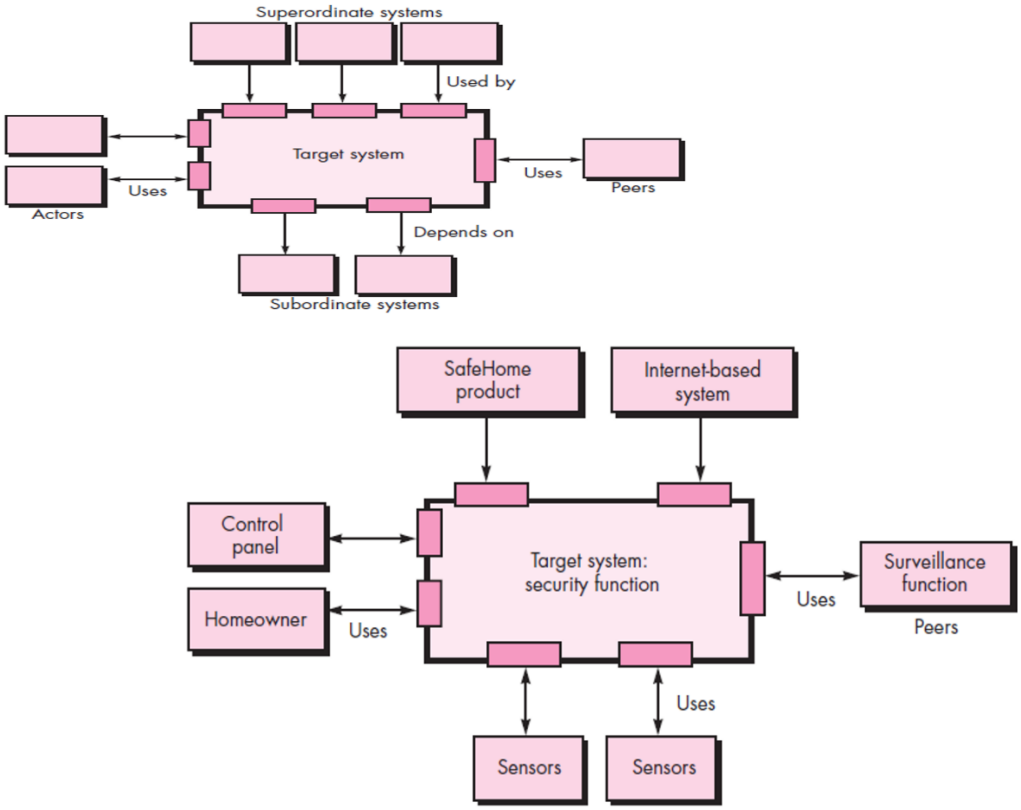


Figure: Architectural context diagram for the SafeHome security function

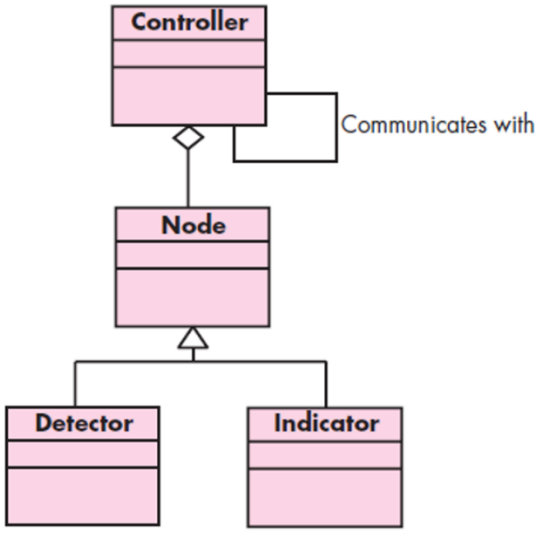


Figure: UML relationships for SafeHome security
Function archetypes

Class Diagram

Shows the static structure of classes and their relationships

Library Management System - Class Diagram

```
                          Library
- name: String
- address: String
- books: List<Book>
- members: List<Member>
+ addBook(book: Book): void
+ removeBook(isbn: String): void
+ registerMember(member: Member): void
+ searchBook(title: String): Book
                           Book
- isbn: String
- title: String
- author: String
- isAvailable: boolean
+ getDetails(): String
+ setAvailability(status: boolean): void
+ isAvailable(): boolean
                          Member
- memberId: String
- name: String
- email: String
- borrowedBooks: List<Book>
+ borrowBook(book: Book): boolean
+ returnBook(book: Book): void
+ getBorrowedBooks(): List<Book>
                  Library1 ──────── *Book
                 Library1 ──────── *Member
                  Member* ──────── *Book
```

Class Diagram Elements

Class Structure

- **Class Name:** Top compartment, centered, bold
- **Attributes:** Middle compartment, format: visibility name: type
- **Methods:** Bottom compartment, format: visibility name(params): returnType

Visibility Modifiers

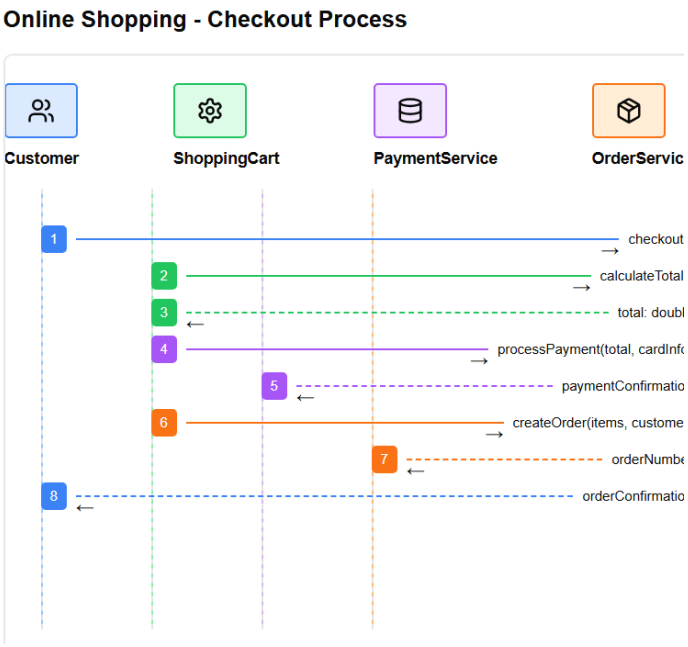
- **+ (Public):** Accessible from anywhere
- **- (Private):** Accessible only within the class
- **# (Protected):** Accessible within class and subclasses
- **~ (Package):** Accessible within the same package

Relationships

- **Association:** ——————— (simple line)
- **Aggregation:** ◇——————— (hollow diamond)
- **Composition:** ♦——————— (filled diamond)
- **Inheritance:** ———————▷ (hollow triangle)
- **Dependency:** - - - - - → (dashed arrow)
- **Realization:** - - - - - ▷ (dashed line with triangle)

Multiplicity

- **1:** Exactly one
- **0..1:** Zero or one
- ***:** Zero or more
- **1..*:** One or more
- **n:** Exactly n
- **n..m:** Between n and m



Sequence Diagram Elements

Basic Elements

- **Actor:** External entity (stick figure or rectangle)
- **Object:** System component (rectangle with underlined name)
- **Lifeline:** Vertical dashed line showing object existence
- **Activation Box:** Rectangle on lifeline showing active period

Message Types

- **Synchronous:** ——————→ (solid line with filled arrow)
- **Asynchronous:** ——————→ (solid line with open arrow)
- **Return:** - - - - - → (dashed line with arrow)
- **Self-call:** Loop back to same lifeline
- **Create:** ——————→ <<create>>
- **Destroy:** ——————→ <<destroy>>

Control Structures

- **alt:** Alternative execution (if-else)
- **opt:** Optional execution (if)
- **loop:** Repeated execution (while/for)

- **par:** Parallel execution
- **seq:** Sequential execution
- **critical:** Critical section

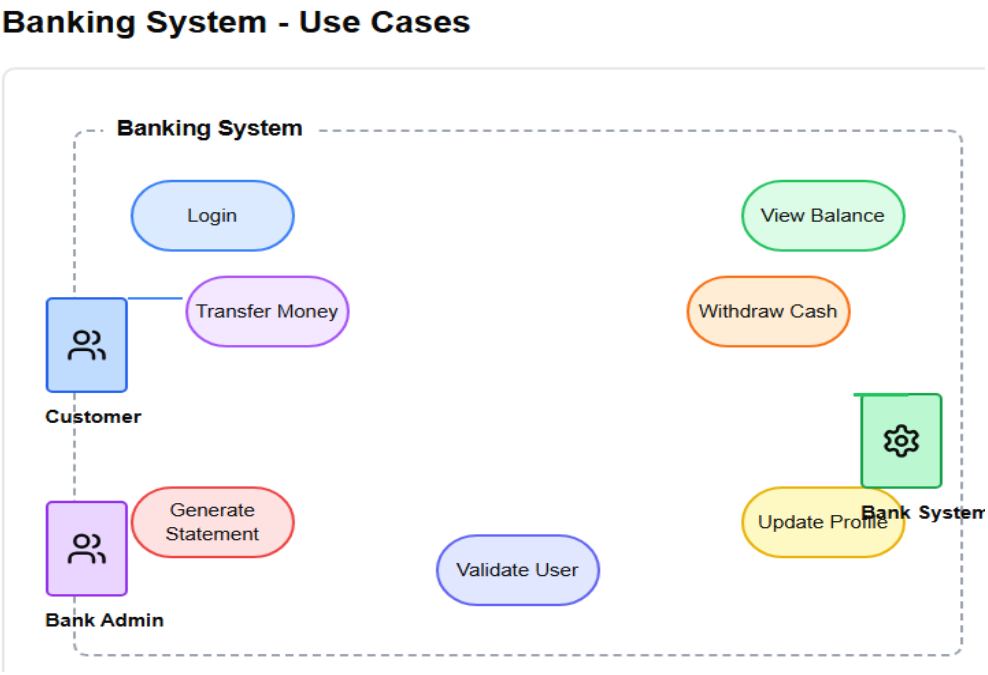
Best Practices

- • Number messages sequentially
- • Keep diagrams focused on specific scenarios
- • Show return messages for clarity
- • Use activation boxes to show processing time
- • Include guards and conditions when necessary

Use Case Diagram

Shows system functionality from user's perspective

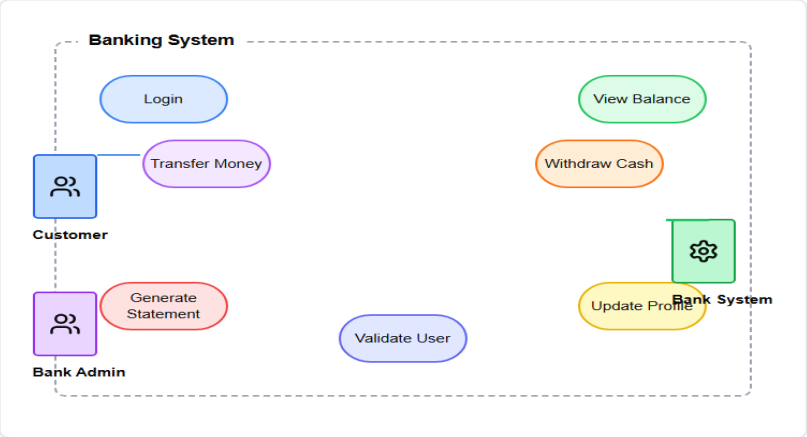
Banking System - Use Cases



Use Case Diagram

Shows system functionality from user's perspective

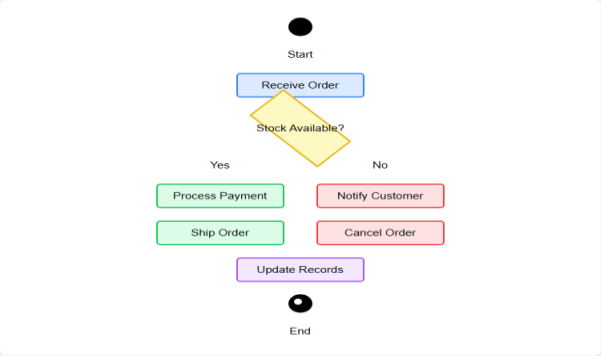
Banking System - Use Cases



Activity Diagram

Shows workflow and business process flow

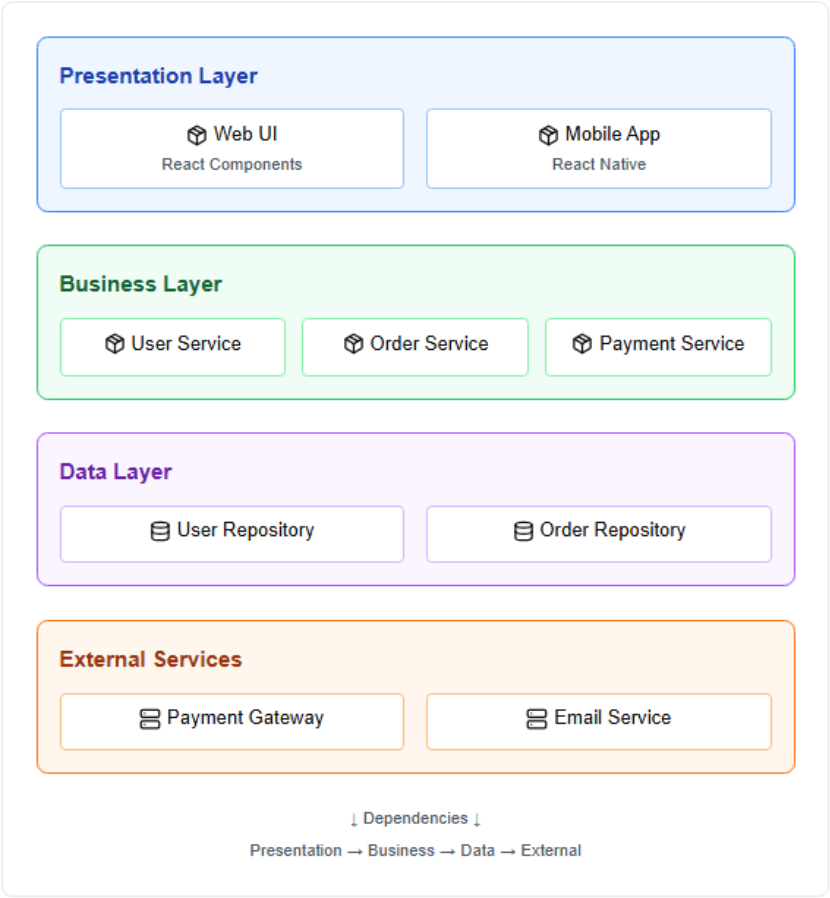
Order Processing Workflow



Component Diagram

Shows software components and their dependencies

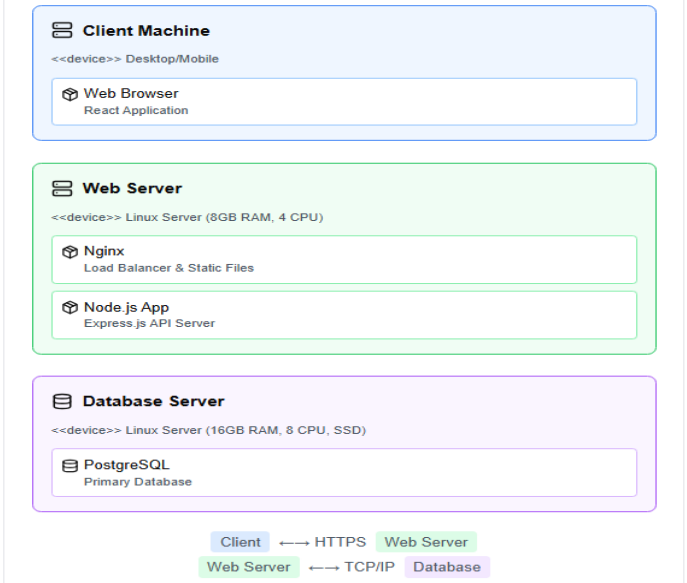
E-commerce System Components



Deployment Diagram

Shows physical deployment of software components on hardware

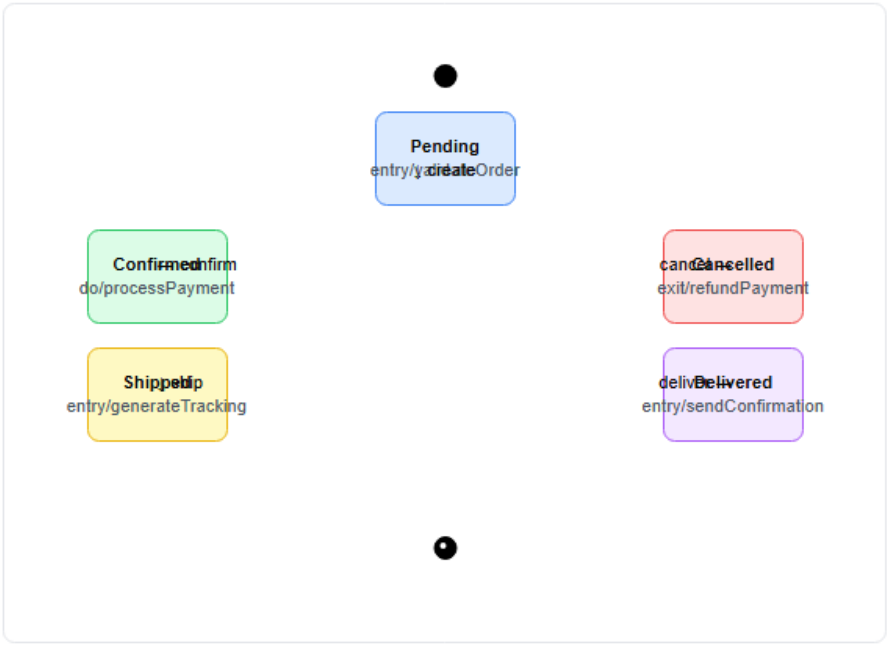
Web Application Deployment



State Diagram

Shows states and transitions of an object or system

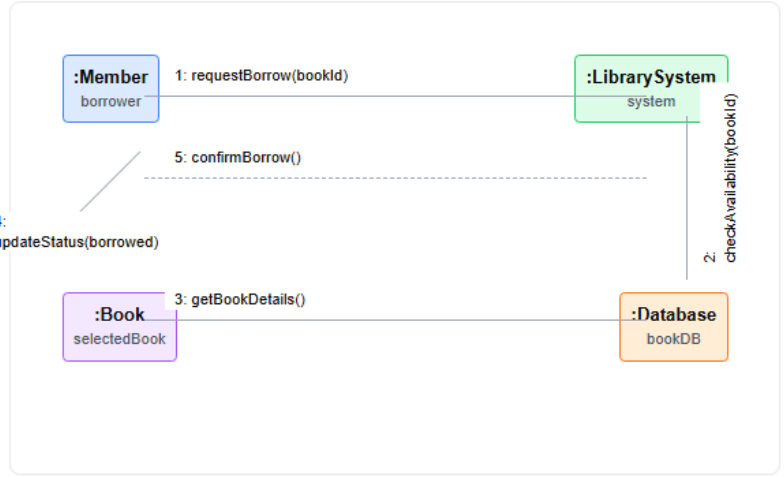
Order State Machine



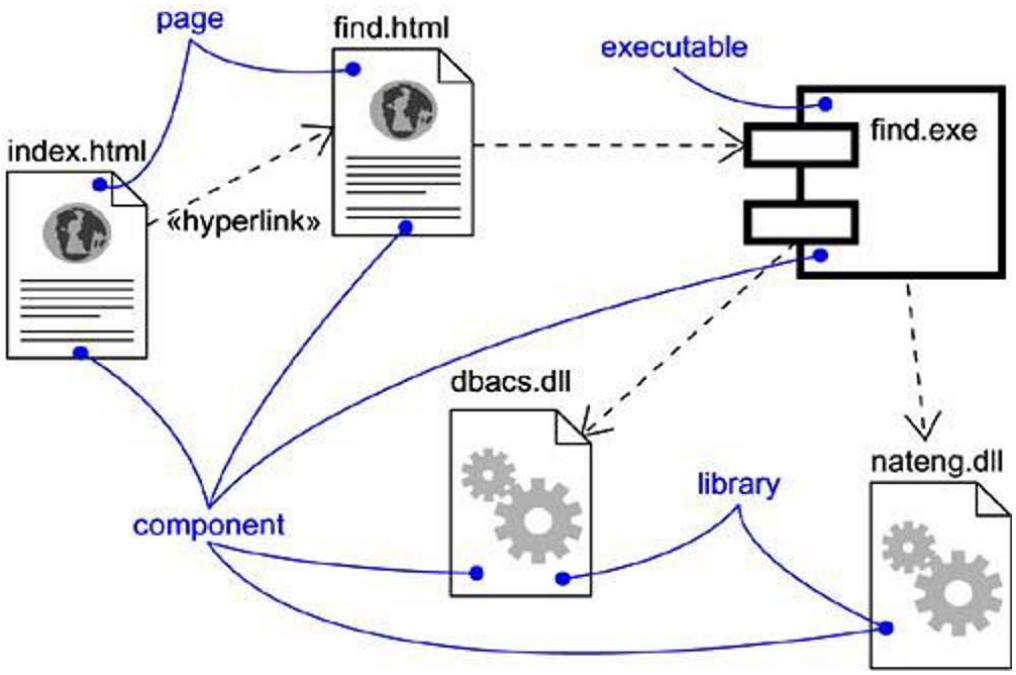
Collaboration Diagram (Communication Diagram)

Shows interactions between objects emphasizing structural organization

Library Book Borrowing System



Component Diagram



Terms and Concepts

A component diagram shows a set of components and their relationships.
Graphically, a component diagram is a collection of vertices and arcs.
Component diagrams commonly contain
Components
Interfaces
Dependency, generalization, association, and realization relationships

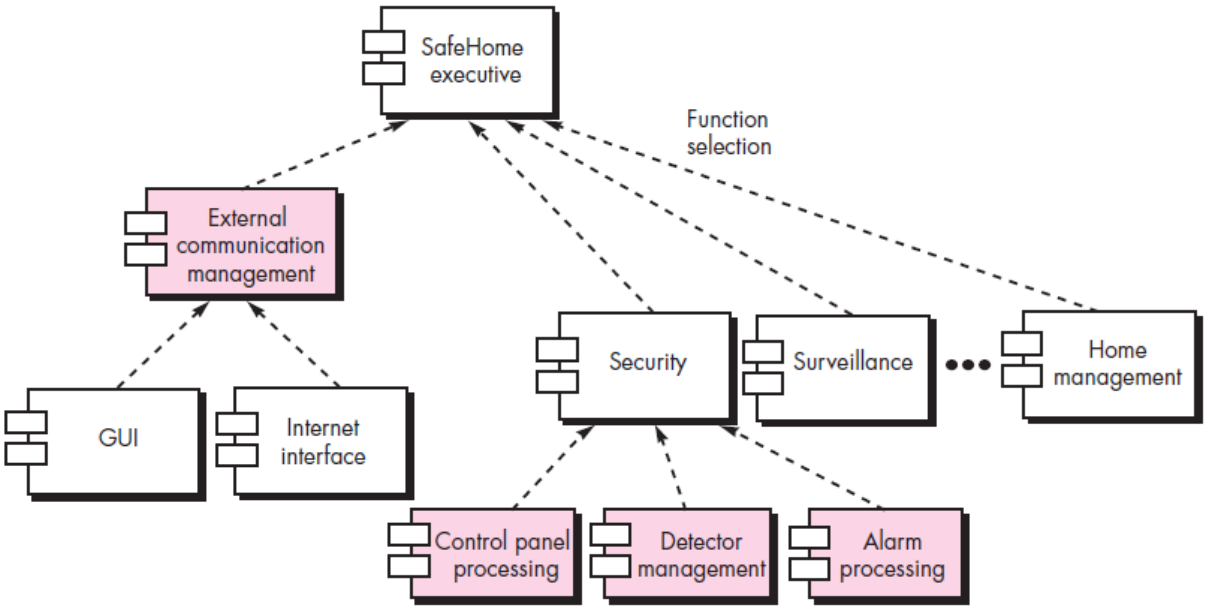


Figure: Overall architectural structure for SafeHome with top-level components